

Visualizing optimization algorithms for deep learning intuition

✓ Good prompt design:
Forces PREDICTION before
visualization. This is key
for active learning.

⑧ You are a visual learning tutor helping me understand for CS182 (graduate deep learning at Berkeley). Your goal is to help me BUILD INTUITION through interactive visualizations, not just equations.
RULES:

1. For every optimizer concept, generate **RUNNABLE** Python code that visualizes it
 2. Before showing math, show me what the optimizer landscape
 3. Ask me to **PREDICT** what will happen before running the visualization
 4. After I see the visualization, ask me to explain **WHY** it behaved that way
 5. Build up complexity: start with vanilla SGD, then add momentum, then Adam
- STRUCTURE: Phase 1: Setup a 2D Loss Landscape
- Generate code for a challenging 2D loss surface (e.g. a saddle or a ravine)
 - Visualize it as a contour plot
 - Ask me: "Where do you think gradient descent will struggle on this surface? Why?"
- Phase 2: Vanilla SGD
- Animate SGD trajectory on the loss landscape
 - Show the path it takes, step by step
 - Ask me to explain what I observe (oscillation? slow convergence? why?)
- Phase 3: SGD + Momentum
- Same landscape, now with momentum
 - Ask me to **PREDICT** how the trajectory will differ **BEFORE** showing me
 - Animate it, then ask: "Why did momentum help/hurt here?"
- Phase 4: Adam
- Visualize the velocity vector at each step
 - Same landscape, now with Adam

The phase structure builds
complexity gradually:
SGD → Momentum → Adam

Requiring **RUNNABLE** code
means I can tweak params
and experiment myself.



- Ask me to predict the difference from momentum
- Animate it, show the adaptive learning rates per dimension
- Ask: "Why does Adam handle the ravine differently?"
Side Comparison
- Generate a single animation showing ALL THREE optimizers racing on the same landscape
- Different colors for each trajectory
- Ask: "In what situations would you choose each optimizer?" Phase 6: Pathological Cases
- Show me a landscape where momentum HURTS (overshooting)
- Show me a landscape where Adam struggles (shallow minima, poor generalization)
- Ask me to explain each failure mode REQUIREMENTS FOR CODE:
- Use matplotlib with animation (FuncAnimation) or generate frame-by-frame GIFs
- Make code self-contained and runnable in a Jupyter notebook or as a .py file
- Include clear comments explaining each part
- Use a reasonable number of iterations so animations aren't too long (50-200 steps) Context: I understand gradient descent conceptually and know the optimizer update formulas, but I don't have intuition for WHY momentum/Adam behave differently in practice. Start with Phase 1 - generate the loss landscape and ask me your first question.

Phase 6 (pathological cases) is smart — understanding failure modes is as important as understanding success.

Context line tells the AI my current level — I know formulas but lack intuition.

Thought: Architected interactive visualization framework for optimization algorithm intuition-building.

Phase 1: Building Your Loss Landscape Intuition



Let's start by creating a challenging 2D loss surface that will expose the weaknesses and strengths of different optimizers.

I'll use the **Beale function** - it's a classic optimization test function with a narrow, curved valley that creates real problems for optimizers. It has steep walls and a long, winding path to the minimum.

AI chose Beale function — good choice, it has the narrow ravine that exposes optimizer weaknesses.

Create File: Creating Phase 1: 2D Loss Landscape visualization for CS182 optimization tutorial

Thought: Now let me run this script to generate the visualizations.

Bash Tool: Running Phase 1 loss landscape visualization script

I like that it generates code first, shows me the landscape, THEN asks questions.

Thought: Good, the script ran successfully. Now I'll move the output files to the outputs directory so the user can view them.

Bash Tool: Copy Python script to outputs for user access

Thought: Prepared to deliver generated visualizations to user.

View: Viewing the generated loss landscape contour plot

Thought: Orchestrated visualization sequence and prepared gradient field demonstration.

View: Viewing the gradient field visualization

Thought: Orchestrated visualizations and formulated opening question for interactive learning.

Here's your loss landscape, Rishi! Take a moment to study both visualizations:

Figure 1: Contour + 3D View

 Loss Landscape

The gradient field viz (arrows) was super helpful. I could SEE why SGD would struggle before running it.

Figure 2: Gradient Field (arrows show steepest descent direction)

 Gradient Field

[View the runnable code](#)

Good setup table explaining what each element means. No ambiguity.

The Setup

This is the **Beale function** - a classic optimization test case. Here's what you're looking at:

Element	Description
★ Red star	Global minimum at (3, 0.5)
○ White circle	Starting point at (0, 1.5)
Red arrow	Steepest descent direction at start
Colors	Log-scaled loss (purple = low, yellow = high)

✓ Prediction task asks specific questions, not just 'what do you think'.

The contour lines are **NOT** circles - they're elongated ellipses forming a **curved ravine** that leads to the minimum.

 Your First Prediction Task

Before we run any optimizer, I want you to look carefully at the gradient field (Figure 2) and answer:

Question: Starting from the white circle at $(0, 1.5)$, where SGD (constant learning rate, no momentum) will **struggle**

My prediction about oscillation came from looking at the gradient field — visual → intuition!

Think about:

1. What happens when the gradient points strongly in one direction but weakly in another?
2. What do you notice about the arrows near the valley floor vs. on the walls?
3. If you take steps proportional to the gradient magnitude, what problem might arise?

Take your best guess and explain your reasoning - I'll show you the actual SGD trajectory in Phase 2 to test your intuition!

I identified the key issue: gradient across valley >> gradient along valley. This is the core problem.

👤 Looking at the gradient field, I notice a few things:

1. The arrows near the start point $(0, 1.5)$ are pointing mostly leftward/downward — so SGD will initially head that direction. But the arrows aren't pointing directly at the minimum, they're pointing more "across" the valley.
2. Near the valley floor (the purple region), the arrows seem to point in different directions on opposite sides — like they're pointing "into" the valley from both walls. So if SGD overshoots, it'll get pushed back, then overshoot again... oscillation.
3. The gradient magnitudes look really different — huge on the steep walls (long arrows) and tiny near the valley floor (short arrows). So SGD will take big steps when it's on the walls and tiny steps when it finally gets to the valley. My prediction: SGD will zig-zag down toward the valley pretty fast, but then once it's in the ravine, it'll oscillate back and forth across the narrow valley instead of moving along it toward the minimum. The gradients across the valley are much larger than the gradients along it, so most of the step size gets "wasted" on the oscillation direction. It

might eventually get to the minimum but it'll be really inefficient — lots of steps, most of them just bouncing side to side. Is that right? Can you show me the actual trajectory?

✓ AI validates my reasoning and sharpens terminology: 'gradient misalignment', 'magnitude mismatch'.

Thought: Validated user's gradient intuition and planned SGD visualization.

That's an *excellent* analysis! You've correctly identified the three key pathologies:

1. **Gradient misalignment** — the gradient points perpendicular to the valley, not toward the minimum
2. **Oscillation dynamics** — opposing gradients on valley walls create a ping-pong effect
3. **Magnitude mismatch** — huge gradients across the valley along it

Good that it names the three pathologies explicitly. Gives me vocabulary.

Let's see if your intuition holds up! Here comes Phase 2:

Phase 2: Vanilla SGD in Action

Create File: Creating Phase 2: Animated SGD trajectory visualization

Thought: Prepared to execute visualization script.

Bash Tool: Running Phase 2 SGD animation



Thought: Identified unnecessary import and resolved code cleanup.

Edit: Remove IPython import that's not needed

Thought: Prepared to install dependencies and trajectory results.

AI generates actual
runnable code — not
pseudocode. I can copy
and run this myself.

Bash Tool: Install pillow and run Phase 2 animation

Thought: Reconsidered initial setup to better demonstrate oscillation behavior.

Interesting! The SGD went the *wrong direction* because the gradient at our starting point actually points away from the minimum. This is a great teaching moment, but let me adjust the starting point to better demonstrate the oscillation behavior you predicted:

Edit: Change starting point to better demonstrate oscillation in ravine

Thought: Refined visualization parameters and axis scaling for trajectory display.

Bash Tool: Re-run with adjusted starting point

Thought: Recalibrated starting point and learning rate to better demonstrate oscillation.

Good progress, but let me adjust the setup to show more dramatic oscillation:

Edit: Adjust parameters for clearer oscillation visualization

Thought: Identified need to correct starting point in summary.

△ Interesting: AI had to debug and adjust params. Shows the iterative nature of getting good visualizations.

Edit: Fix the summary to show correct starting point

Thought: Recalibrated distance metrics from revised starting coordinates.

Edit: Fix efficiency calculation to use actual starting point

Thought: Now let me run the improved version.

It transparently shows when things don't work and explains why — good for learning process.

Bash Tool: Run improved SGD visualization

Thought: Reconsidered optimization demonstration strategy and adjusted parameters.

Hmm, SGD diverged in the wrong direction! This is actually an important lesson - the Beale function has regions where the gradient points away from the minimum. Let me use a cleaner example that shows oscillation perfectly, then return to Beale:

Create File: Creating Phase 2 with ill-conditioned quadratic to clearly show oscillation

Thought: Prepared to demonstrate gradient descent trajectory visualization.

Bash Tool: Run improved Phase 2 with quadratic demo

Thought: Validated quadratic demo success, pivoted to alternative approach.

AI pivots to simpler quadratic when Beale diverges. Smart pedagogy — start simple, then complex.

The quadratic demo worked perfectly - **99 oscillations**! The Beale function diverged (demonstrating instability - another important lesson), but the quadratic shows exactly what you predicted. Let me fix Beale and show you the results:

Edit: Fix Beale function with smaller learning rate

Thought: Prepared to rerun analysis with corrected parameters.

Bash Tool: Run fixed version and copy to outputs

Thought: Now let me view the images.

View: View the quadratic SGD visualization

Thought: Visualizing the trajectory and examining the animation.

View: View the Beale SGD visualization

Thought: Good results. Now let me also view the animation GIF. Then I'll summarize the findings for the user.

View: View the SGD animation GIF

Thought: Synthesized gradient analysis and prepared trajectory

visualization.

🎯 My prediction confirmed!
99 oscillations across valley.
Seeing the actual trajectory
made it concrete.

🎯 **Your Prediction Was SPOT ON!**

Your results:

Part A: Ill-Conditioned Quadratic (the cleaner demo)

 Quadratic SGD

Part B: Beale Function (the realistic challenge)

The metric table is great:
Prediction vs Actual Result.
Clear feedback loop.

 Beale SGD

Animated Version (open in browser to see it move!)

 SGD Animation

[View the Python code](#)

What the Data Shows

Metric	Your Prediction	Actual Result
Oscillation	"Zig-zag across valley"	✅ 99 y-direction reversals!
Progress along valley	"Slow"	✅ x goes from 10→0.48 (100 steps, still not at 0)

Metric	Your Prediction	Actual Result
Gradient imbalance	"Wasted on oscillation direction"	 y-gradient is 50x larger than x-gradient The 'learning rate dilemma' ASCII diagram is a nice way to summarize the core tradeoff.

Look at the bottom-left panel: the y-coordinate oscillates violently at first, then settles. Meanwhile, the x-coordinate (bottom-right) decreases smoothly but *painfully slowly*.

The Core Problem You Identified

Learning rate dilemma:

- └─ lr too big → y oscillates, diverges
- └─ lr too small → x makes glacial progress
- └─ ANY **single** lr → *can't satisfy both dimensions!*

✓ Momentum prediction questions are specific:
 1. Oscillation?
 2. Progress?
 3. New failure modes?



Phase 3 Preview: Momentum

Now here's my question for you before we visualize momentum:

Momentum adds "velocity" that accumulates across steps:

$$\begin{aligned}
 v_{t+1} &= \beta * v_t + \nabla f(\theta_t) \\
 \theta_{t+1} &= \theta_t - lr * v_{t+1}
 \end{aligned}$$

PREDICTION TIME:

On the same ill-conditioned quadratic, how do you think momentum will change the trajectory? Specifically:



1. **Oscillation behavior:** Will momentum make the y-direction oscillation *better* or *worse*? Why?
2. **Progress along valley:** Will momentum help the x-direction progress *faster* or not? Why?
3. **Where might momentum cause NEW problems?**

AI confirmed my SGD prediction before moving on. Good reinforcement.

Think physically: momentum is like a ball rolling down a hill that builds up speed. What happens when a fast-rolling ball enters a narrow valley? What happens when it reaches the valley floor and keeps rolling?

Take a guess, then I'll show you!

👤 Looking at the gradient field — the arrows near the valley point "into" the walls from both sides, not along the valley toward the minimum. My prediction: SGD will zig-zag down to the ravine quickly, but then just oscillate back and forth across it instead of moving along it to the minimum. The gradient is way stronger across the valley than along it. Show me the trajectory?

Thought: Reconciled user's SGD prediction with Phase 3 momentum question.

You already nailed the SGD prediction! I showed you the results above — **99 oscillations** across the valley, exactly as you predicted. Your intuition is solid. 🎯

Now let's level up. Before I show you **Phase 3: Momentum**, I need your prediction:



Momentum Prediction Challenge



Momentum modifies SGD by adding "velocity" that accumulates:

$$v_{t+1} = \beta \cdot v_t + \nabla f(\theta_t) \quad \leftarrow \text{velocity accumulation}$$
$$\theta_{t+1} = \theta_t - \text{lr} \cdot v_{t+1} \quad \leftarrow \text{move by velocity}$$

gradient

The 'heavy ball' physical analogy helps me think about momentum intuitively.

Think of it like a heavy ball rolling downhill — it builds up speed and has inertia.

Your task: Predict what happens on the same ravine landscape

Question 1: Oscillation across the valley (y-direction)

The y-gradient keeps flipping sign (positive → negative → positive) → bounces wall-to-wall. With momentum accumulating velocity...

Specific hints guide my thinking without giving away the answer.

- Will oscillations get **dampened** (smaller over time)?
- Will oscillations get **amplified** (larger over time)?
- Why?

Question 2: Progress along the valley (x-direction)

The x-gradient is consistently small but always points the same direction (toward minimum). With momentum accumulating...

- Will x-progress be **faster** than vanilla SGD?
- Will it be **about the same**?
- Why?

Question 3: Any new failure modes?



What could go *wrong* with a heavy ball rolling through a narrow valley? Think about what happens when it reaches the minimum...

My 3 predictions:
1. Dampened oscillation
2. Faster x-progress
3. Overshooting risk
All based on physics intuition!

Give me your predictions, then I'll run the visualization and we'll see if momentum is the hero or the villain here!

👤 My Response: Q1 - Oscillation: Dampened. The y-velocity keeps flipping sign each bounce, so with $\beta < 1$ the positive and negative components partially cancel out over time. The ball "settles" instead of bouncing forever. Q2 - X-progress: Faster. The x-gradient is small but always points the same way, so momentum accumulates in that direction. Small consistent pushes build up to actual speed along the valley. Q3 - Failure mode: Overshooting. If the ball builds up too much speed rolling down, it might blast past the minimum and climb up the other side. Could oscillate around the minimum or even diverge if learning rate is too high. Let's see it!

Thought: Validated momentum predictions and architected comprehensive visualization framework.

Excellent predictions! Let's put them to the test.

Phase 3: SGD with Momentum

Create File: Creating Phase 3: Momentum visualization comparing to vanilla SGD

Thought: Prepared to execute momentum visualization demonstration.

Bash Tool: Run Phase 3 momentum visualization

Thought: Excellent! All three predictions were correct:

1. Oscillations dampened: 99 → 43
2. X-progress faster: 0.4755 → -0.0248 (much c
3. Overshoot: Yes, it overshoot to -3.7935!

AI generates comparison
code showing BOTH SGD
and momentum side-by-side.
Direct comparison is key.

Now let me view the generated images to show the user.

View: View the trajectory comparison

Thought: Interesting! The momentum trajectory shows more dramatic motion - you can see it overshoots past the minimum and comes back. SGD stays tightly along the valley floor after the initial drop. Let me show the detailed analysis.

View: View the detailed analysis with coordinate plots

Thought: Analyzing momentum's oscillation dampening and acceleration mechanics.

View: View velocity components

Thought: Validating predictions through momentum visualization.

View: View velocity vectors visualization

Thought: Analyzing velocity magnitude shifts and overshooting dynamics.

View: View the race animation GIF

Thought: Synthesized physics concepts and ready visualization presentation.

🎯 3/3 predictions correct!
Oscillations: 99 → 43
X-progress: 20x closer
Overshoot: yes, to $x=-3.8$

🎯 Phase 3 Results: You Nailed It 3/3!

Your Predictions vs Reality

Trajectory Comparison

 Trajectories

The velocity components plot is the 'money shot' — shows WHY momentum works.

Detailed Analysis

 Analysis

The Key Insight: Velocity Components

 Velocity

Velocity Vectors Over Time

 Vectors

Animated Race (open in browser)

[View the Python code](#)

 **KEY INSIGHT:**
Along valley: same sign
→ velocities ACCUMULATE
Across valley: flipping sign
→ velocities CANCEL

Scorecard

Prediction	Your Answer	Result
1. Oscillation dampened?	"Yes - velocities cancel"	✅ 99 → 43 oscillations
2. X-progress faster?	"Yes - velocities accumulate"	✅ x: 0.48 → -0.02 (20x closer!)
3. Overshooting?	"Yes - ball builds speed"	✅ Overshot to x = -3.8!

This table summary is exactly what I needed. Now I understand WHY momentum helps.

The Velocity Plot Tells The Whole Story

Look at that velocity components plot — it's the visual proof of your intuition:

Left (x-velocity): Smooth buildup to $v=40$, then oscillates as it overshoots and returns. The velocity is *mostly the same sign* so it **accumulates**.

Right (y-velocity): Wild oscillation between $+40$ and -40 , but the alternating signs mean they **partially cancel**. The envelope shrinks over time.

Why Momentum Works (Your Summary Was Perfect)

Direction

Gradient Sign

Velocity Effect

Result

Along valley	Same each step	ACCUMULATES
Across valley	Flips each step	CANCELS

Adam explanation builds on momentum — shows first moment + second moment. Good scaffolding.



Phase 4 Preview: Adam

Now for the final boss. **Adam** combines:

1. **Momentum** (first moment - like what we just saw)
2. **RMSprop** (second moment - adaptive learning rates)

Hints point me to think about the denominator $\sqrt{\hat{v}_t}$ without giving away answer.

```
m_t = β1 * m_{t-1} + (1-β1) * g_t           # First moment
(momentum-like)

v_t = β2 * v_{t-1} + (1-β2) * g_t2         # Second moment
(gradient variance)

θ_t = θ_{t-1} - lr * m_t / (√v_t + ε)         # Adaptive update
```

PREDICTION TIME:

On the same ill-conditioned quadratic:

1. **How will Adam handle the y-direction differently than momentum?**

- Hint: v_t tracks the *squared* gradient. What's special about y's gradient?

2. **Will Adam overshoot like momentum did?**

- Hint: The denominator $\sqrt{\hat{v}_t}$ scales the step size. What happens when you've been seeing large gradients?



3. What's the tradeoff?

- If Adam "knows" the y-direction has big gradients and scales them down, is that always good?

My prediction about adaptive LR came from understanding the formula:
large $g^2 \rightarrow$ large $\sqrt{\hat{v}} \rightarrow$ small step

Think about it, then tell me your predictions!

- ⊙ My Response: Q1 - Y-direction: The y-gradient is huge but oscillates. v_t (second moment) tracks g^2 so it'll see "wow, y has massive gradients" and divide by a large $\sqrt{\hat{v}}_t$. This shrinks the y step size automatically — Adam learns to take smaller steps in the bouncy direction. Q2 - Overshooting: Less than momentum. The adaptive denominator acts like automatic learning rate decay for directions with large gradients. When you've been seeing big gradients, you take more cautious steps. Q3 - Tradeoff: If Adam aggressively shrinks steps in high-gradient directions, it might be too cautious and converge slower than momentum in some cases. Also, the "memory" of past gradients (β_2) might not adapt fast enough if the landscape is near sharp minima or when entering a new region. So

I predicted the tradeoff:
Adam might be TOO cautious.
This turned out to be true!

Thought: Architected comprehensive visualization testing three optimizer predictions.

Your intuition keeps getting sharper! Let's put Adam to the test.

Phase 4: Adam — The Adaptive King

Create File: Creating Phase 4: Adam visualization comparing all three optimizers

Thought: Synthesized gradient behavior and optimization tradeoffs comprehensively.

Bash Tool: Run Phase 4 Adam visualization

Triple comparison now:
SGD vs Momentum vs Adam
all on same landscape.

Thought: Analyzed optimizer performance metrics visualizations.

View: View triple trajectory comparison

Thought: Validated user's optimizer analysis through trajectory comparison.

View: View the adaptive learning rate analysis - the key insight

Thought: Validated user's predictions about adaptive learning rates and gradient behavior.

View: View coordinate comparison

Thought: Analyzed optimizer behavior across convergence metrics and gradient dynamics.

View: View step size equalization

Thought: Examined visualization nuances and contemplated animation insights.

View: View the triple race animation

Thought: Synthesized optimizer mechanics and prepared comparative analysis results.

🎯 Again 3/3 correct!
The effective LR plot shows x gets 3.3x higher LR than y. Visual proof!

🎯 Phase 4 Results: Perfect 3/3 Again!

The Trajectories Tell The Story

 Triple Trajectories

Look at the difference! Adam (right) takes a *smooth, controlled* path while momentum (middle) goes on a wild ride.

🎯 'Money shot' = effective learning rate plot.
Green (x) stays high, Blue (y) drops. Adam adapts!

The Key Insight: Adaptive Learning Rates

 Adaptive LR Analysis

Top-right panel is the money shot! Look at those effective learning rates:

- **Green (x-direction):** LR stays relatively high → aggressive progress
- **Blue (y-direction):** LR drops dramatically → cautious steps

Bottom-left: v_t (second moment) shows WHY — the y -gradient² is 10x larger, so $\sqrt{v_t}$ is larger, so effective LR is smaller.

Oscillation & Overshooting Comparison

 Coordinate Comparison

Final comparison table:
Oscillations: 99→43→14
Adam wins on stability.

Metric	SGD	Momentum	
Y-oscillations	99	43	14 
Min x (overshoot)	0.48	-3.79	-1.30 
Final loss	0.113	0.0003	0.0014

The Step Size Equalization Effect

 Step Sizes

Step size equalization plot shows Adam's steps are comparable in both directions despite 50x gradient diff!

This is beautiful! Despite the y-gradient being **50x larger** than x-gradient, Adam's actual step sizes in both directions are **comparable**. The adaptive denominator does its job perfectly.

Triple Race Animation

 Race Animation

[View the Python code](#)

Your Scorecard

Prediction	Your Answer	Result
1. y-LR shrinks?	"v_t sees huge y-gradients, divides by large $\sqrt{\hat{v}_t}$ "	x gets 3.3x higher effective LR than Adam INTERESTING FINDING: Momentum got LOWER final loss than Adam (0.0003 vs 0.0014). Exploration helps!
2. Less overshooting?	"Adaptive denominator = automatic caution"	-1.3x
3. Tradeoff?	"Might be too cautious, slow to adapt"	Interesting: momentum got lower final loss!

The Tradeoff You Spotted

You correctly identified a real issue! Look at the final losses:

Momentum: 0.000338 ← Lower!
 Adam: 0.001359 ← 4x higher

This is the tradeoff I predicted! Adam's caution = less exploration. Not always better.

In this specific case, momentum's aggressive overshooting actually helped it explore and settle closer to the minimum. Adam's caution prevented the wild oscillations but also made it slightly less "exploratory."

This is the **exploration-exploitation tradeoff** in optimizers!

Phase 5 & 6: The Grand Finale

We've built up great intuition. Now let's see:

1. **Phase 5:** All three optimizers racing on the **Beale function** (the curvy ravine from Phase 1)

2. **Phase 6:** Pathological cases — where does each optimizer **fail**?

Want me to generate these final visualizations? I'll show you...

Good setup for Phase 5-6:
pathological cases where
each optimizer fails.
Would strengthen understanding.

- A landscape where **momentum's overshooting** is c
- A landscape where **Adam's "memory" becomes a liability**

Ready for the finale?

Overall: The predict→
visualize→verify loop
worked really well.
I built real intuition.